

PROJECT REPORT: PERCEPTRON USING OOP

Team- Team++

1. Project Description

We made a Perceptron using OOP which can learn the basic AND, OR logic gates. The perceptron takes two inputs x_1 and x_2 , and calculates a linear weighted sum using weights and a bias.

This value is passed through an activation function to produce the predicted output. The project provides two activation functions: Step Activation and Sigmoid Activation.

Initially, the perceptron is given random weights and bias. It compares its predicted output with the expected output from the truth table and updates the weights and bias based on the error.

This process is repeated for each input in every epoch until the perceptron correctly predicts all the outputs and converges.

The project is divided into classes such as Activation, StepActivation, Sigmoid, Dataset, Perceptron, and PerceptronTrainer. These classes separate the activation functions, truth-table data, prediction, and training responsibilities. The project also demonstrates OOP concepts such as abstraction, inheritance, polymorphism, encapsulation and composition.

2. Class Structure

Class Name	Attributes and Methods
Activation (Abstract Base Class)	Attributes: None Methods: - activate(double x)
StepActivation (Inherits from Activation class)	Attributes: None Methods: - activate(double x)
Sigmoid (Inherits from Activation class)	Attributes: None Methods: - activate(double x)
Dataset	Attributes: - inputs - outputs Methods: - Dataset(string gate) - getInputs() - getOutputs()
Perceptron	Attributes: - weights - bias

	- act Methods: - Perceptron(int inputSize, Activation* activation) - predict() - update() - display() - ~Perceptron()
PerceptronTrainer	Attributes: None Methods: - trainEpoch() - isConverged()

3. Project Files

i. **Activation.cpp** – Implements the Step and Sigmoid activation functions and provides the actual behaviour for the activate() methods.

ii. **Activation.h** – Declares the Activation, StepActivation, and Sigmoid classes. Defines the activation function interface used by the perceptron.

iii. **Dataset.cpp** – Creates the truth-table data for the selected AND or OR gate. Implements methods for accessing the input and expected output data.

iv. **Dataset.h** – Declares the Dataset class and its attributes. Defines methods for accessing the inputs and outputs.

v. **Perceptron.cpp** – Implements perceptron initialization, prediction, and updating.
Also displays the current weights and bias.

- vi. **Perceptron.h** – Declares the Perceptron class and its attributes.
Defines methods for prediction, updating, and displaying the perceptron.
- vii. **PerceptronTrainer.cpp** – Implements the training process for each epoch.
Also checks whether the perceptron has converged.
- viii. **PerceptronTrainer.h** – Declares the PerceptronTrainer class.
Defines the methods used for training and convergence checking.
- ix. **main.cpp** – Contains the main program and user menus.
Handles gate selection, activation selection, predictions, and training interaction.
- x. **Makefile** – Contains commands for compiling the project and also provides the make clean command to remove generated files.
- xi. **README.md** – Provides an overview of the project.
Contains features, requirements and setup instructions.

4. How the Project Works

- i. **Program starts** – main.cpp displays the menu and asks the user to select the **AND** or **OR** gate.
- ii. **Select activation** – The user selects either Step Activation or Sigmoid **Activation**. The selected activation object is passed to the Perceptron.
- iii. **Dataset creation** – The Dataset class creates the four possible input combinations: (0,0), (0,1), (1,0), and (1,1), along with the expected outputs for the selected gate.
- iv. **Perceptron initialization** – The Perceptron creates random weights and bias. It also stores the selected activation function.
- v. **Prediction** – For each input, the perceptron calculates the weighted sum:
$$z = w_1x_1 + w_2x_2 + \text{bias}$$

This value is passed to the activation function to produce the prediction.

vi. **Compare with expected output** – The prediction is compared with the correct output stored in the Dataset. The difference gives the error.

vii. **Update weights and bias** – If there is an error, the perceptron updates its weights and bias using the error and learning rate.

Weight update:

$$w1(\text{new}) = w1(\text{old}) + \text{learning rate} \times \text{error} \times x1$$

$$w2(\text{new}) = w2(\text{old}) + \text{learning rate} \times \text{error} \times x2$$

Bias update:

$$\text{bias}(\text{new}) = \text{bias}(\text{old}) + \text{learning rate} \times \text{error}$$

Where:

- error = target - prediction
- learning rate = η (We used 0.1)
- $x1, x2$ = input values
- $w1, w2$ = weights

viii. **Training by epochs** – PerceptronTrainer processes all four input combinations once in an epoch. The user can either view epochs one by one or display all epochs until convergence.

ix. **Check convergence** – After each epoch, `isConverged()` checks whether all four predictions match their expected outputs.

x. **Final result** – When all predictions are correct, the model is considered **converged**. The final weights, bias, and predictions are displayed.

5. OOP Concepts Used

OOP Concept	How it was used in the project
Encapsulation	Data members such as weights, bias, inputs, and outputs are kept private and accessed through public methods.
Abstraction	Activation is an abstract class containing the pure virtual function activate(), defining a common interface for activation functions.
Inheritance	StepActivation and Sigmoid inherit from the Activation base class.
Polymorphism	The Activation* pointer in Perceptron allows different activation objects, such as StepActivation or Sigmoid, to be used through the same interface.
Function Overriding	StepActivation and Sigmoid provide their own implementations of the activate() function.
Composition / Has-a Relationship	Perceptron has an Activation* member, meaning the perceptron uses an activation object as part of its operation.
Constructors	Constructors initialize objects, such as setting the dataset according to the selected gate and initializing perceptron weights and bias.
Destructors	The Perceptron destructor releases the dynamically allocated activation object.
Getters	getInputs() and getOutputs() provide controlled access to the private dataset attributes.